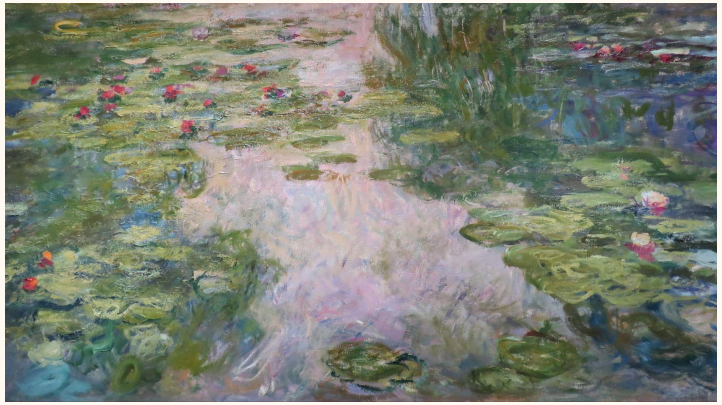




Art Style Classification Models

CSCI 635-01: Intro to ML

Rochester Institute of Technology, Fall 2025

Katelyn Clark & Sabrina Boyle

Why classify art styles?

- Museums, digital archives, etc., are relying more on automatic image tagging
- Art datasets have complex patterns, requiring strong visual feature extraction
- Overall goal is to build models that can classify paintings into one of five art style categories: *Abstract Expressionism, Baroque, Cubism, Impressionism, or Pop Art*

Challenges in Art Style Classification

- Different artists within the same style category may have drastically different techniques &/or subject matters
- Some styles have overlap, & artists may create works that belong to multiple or different categories .. leading to some similarity across styles
  - One of the characteristics of Baroque is its use of highly contrasting lighting; similarly, a characteristic of Impressionism is its focus on the effects of light
- Samples tend to be incredibly detailed, leading to large amounts of data/features & a struggle with using raw pixel values in models

WikiArt Dataset (Chosen Styles)

Images from Kaggle WikiArt dataset of public domain paintings.

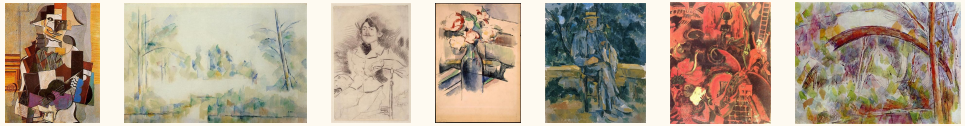
Abstract Expressionism

4.9k

Baroque

8.2k

Cubism



3.5k

Impressionism

16.8k

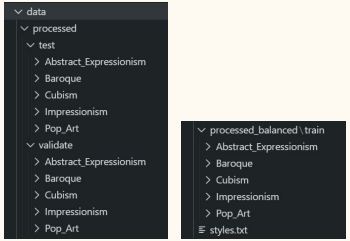
Pop Art

2.9k

Hypotheses

- CNN will outperform the more classical non-NN models
- The most difficult (commonly mixed up) art styles:
  - Baroque & Impressionism
  - Cubism & Pop Art

Preprocessing Steps

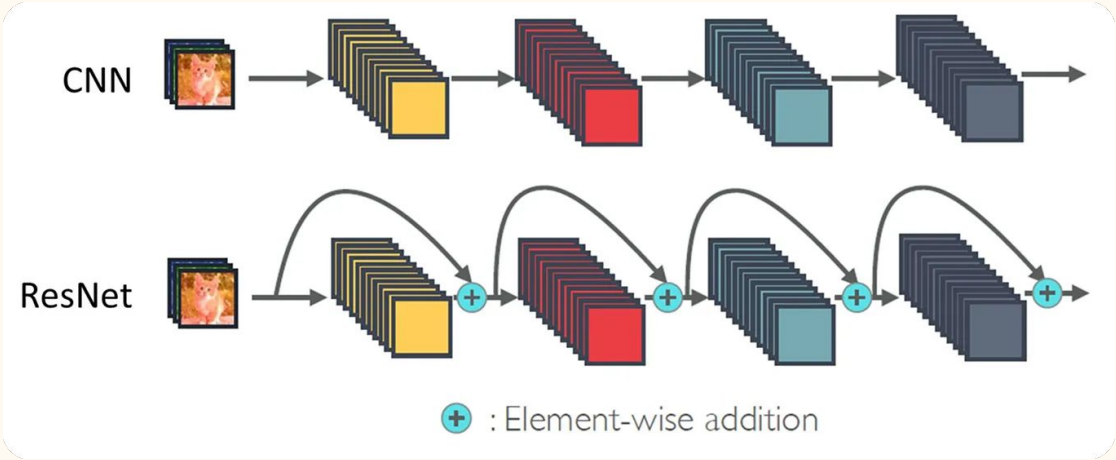


- Split into training/validation/testing with 60/20/20 ratios for each class
- Resized images to 256x256
- Balanced training data .. all training sets get 3,000 images
  - Undersample larger sets
  - Add augmented images to smaller sets via random geometric transformations & color jittering

Original Image | Resized/Downsized | Transformed

(Background Information) Residual Networks

- NN architecture that uses **skip connections** to allow layers to learn a residual function, **preventing vanishing gradients**
  - A skip connection adds the input of the layer directly to its output, creating a **shortcut path**



- **ResNet-50** is a mid-sized residual network known for its efficiency in image classification
  - Can be used to **encode a set of images** by converting the image from RGB to BGR format & then performing a zero-centering operation
  - Extracts **high-quality, discriminative features** that can then be used for other models

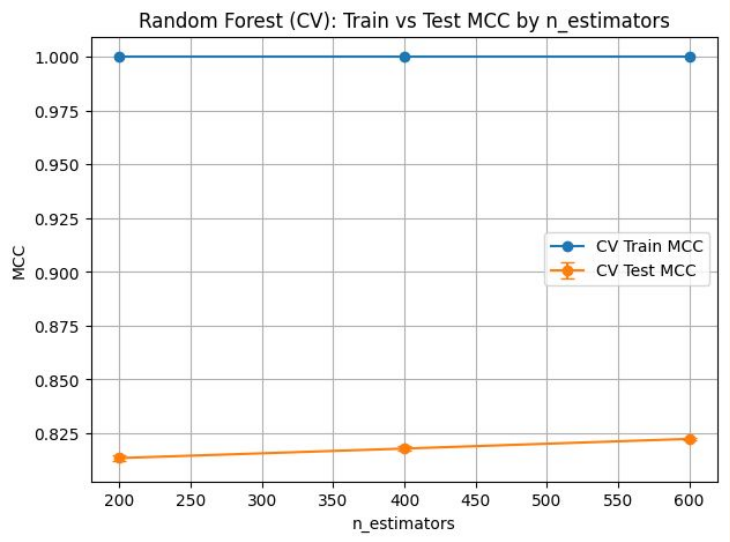
Feature Extraction for Classical Models

- Originally tried to extract features manually in histograms
  - ... poor results with every type of model
- Load images from previously made class folders at 256x256x3
  - Folder name is what determines the image's actual **y**/class value
- Build **tensorflow.data** pipeline: decode, check shape, then batch & prefetch
- Apply **resnet50.preprocess_input** from **tensorflow.keras** to training data
  - ImageNet-style normalization, standard for many computer vision models
- For each image, extract a 2048-dimension feature vector using average pooling

Decision Tree Forest

- **RandomForestClassifier** with **class_weight="balanced_subsample"**
- Hyperparameter search using **GridSearchCV**:
  - n_estimators: [200, 400, 600]
  - max_depth: [None, 20, 40]
  - max_features: "sqrt"
  - min_samples_leaf: [1, 4]
  - Use of PCA: [True, False]
- Cross-validation with **StratifiedKFold**
  - n_splits=2, shuffle=True
- Optimization metric is MCC via **make_scorer(matthews_corrcoef)**
- TEST MCC: 0.7309228607179477 , ACCURACY: 0.8191556395715186

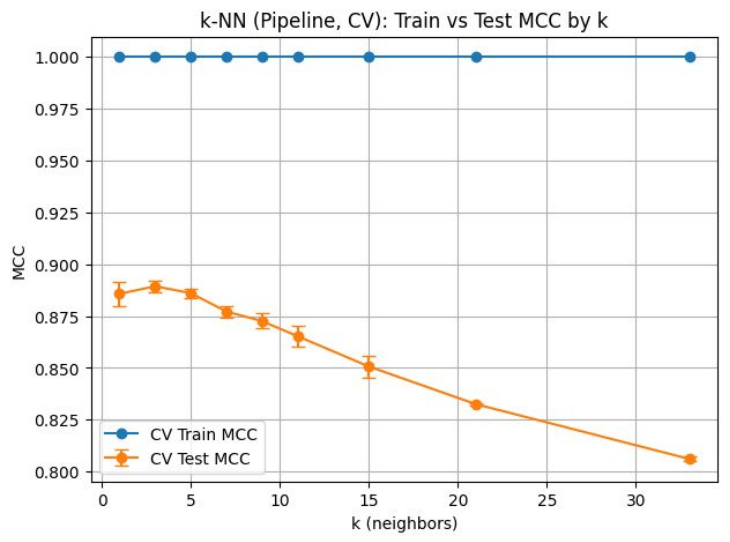
Best parameters: {'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'n_estimators': 600, 'PCA': False} ,
Best CV MCC: 0.8224020849388249



k-Nearest Neighbors

- **KNeighborsClassifier** with **metric="cosine", algorithm="brute"**
- Normalization with **Normalizer(norm="l2")**
- Hyperparameter search using **GridSearchCV**:
  - knn__n_neighbors: [1, 3, 5, 7, 9, 11, 15, 21, 33]
  - knn__weights: [“uniform", "distance"]
- Cross-validation with **StratifiedKFold**
  - n_splits=3, shuffle=True
- Optimization metric is MCC via **make_scorer(matthews_corrcoef)**
- TEST MCC: 0.7479798154669032 , ACCURACY: 0.8271371560596513

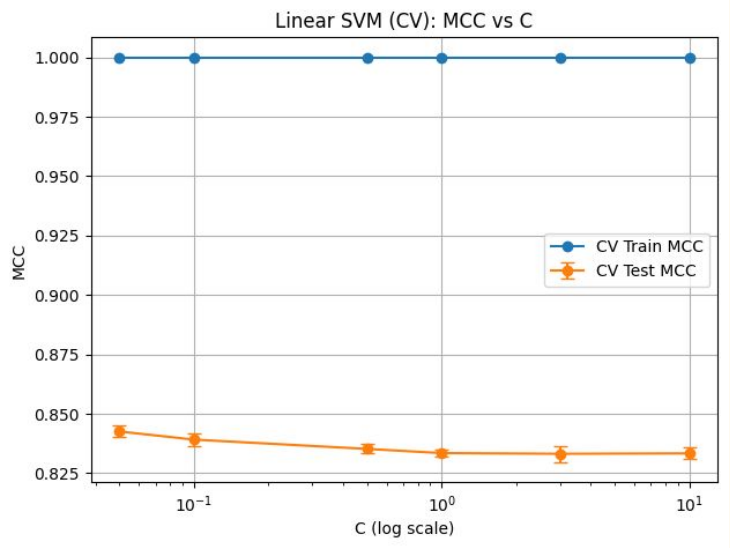
Best parameters: {'knn__n_neighbors': 3, 'knn__weights': 'distance'} ,
Best CV MCC: 0.8893589835643861



Support Vector Machine

- **LinearSVC** with **class_weight="balanced", dual=False, max_iter=5000**
  - Efficient for high-dimensional features
- Standardization using **StandardScaler()**
- Hyperparameter search using **GridSearchCV**:
  - svm__C: [0.05, 0.1, 0.5, 1, 3, 10]
  - svm__tol: [1e-3, 1e-4]
- Cross-validation with **StratifiedKFold**
  - n_splits=3, shuffle=True
- Optimization metric is MCC via **make_scorer(matthews_corrcoef)**
- TEST MCC: 0.7698531201761091 , ACCURACY: 0.8489813064482251

Best parameters: {'svm__C': 0.05, 'svm__tol': 0.0001} ,
Best CV MCC: 0.8425516395756096



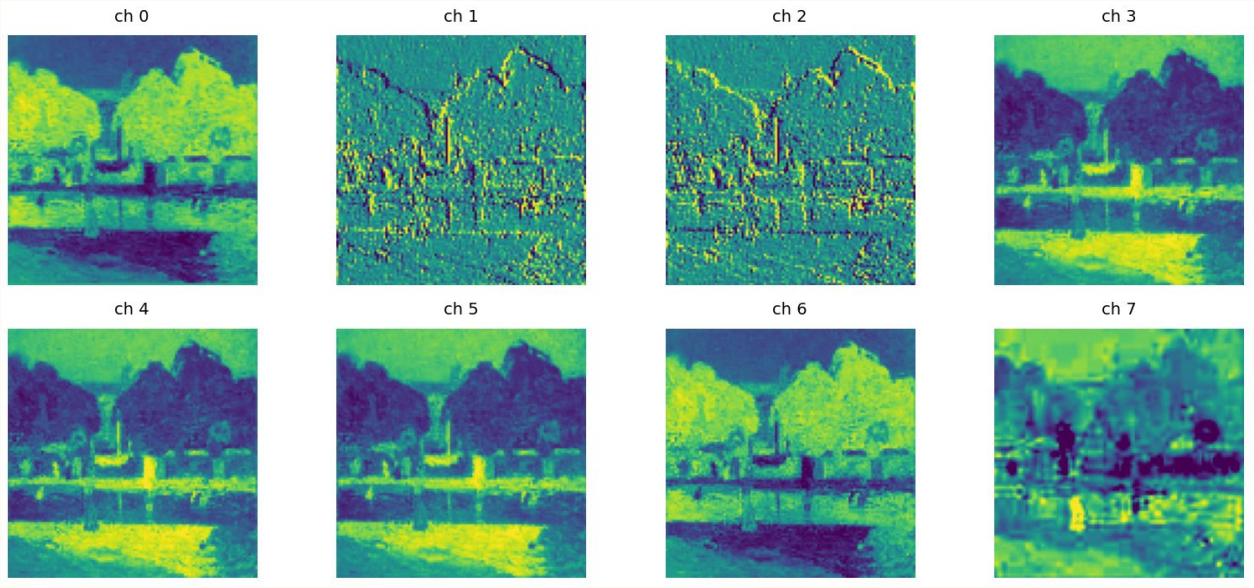
Convolutional Neural Network

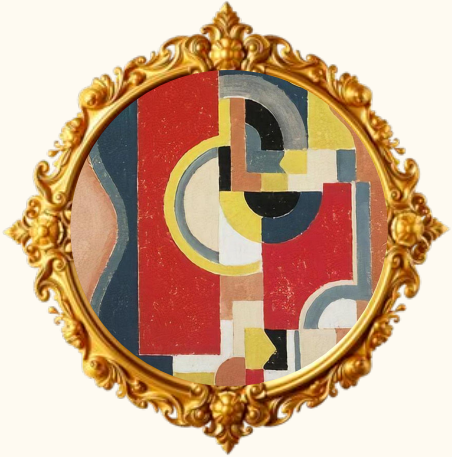
- Doesn't follow the same **resnet50** feature extraction
- **EfficientNetB0** backbone pre-trained on **ImageNet**, frozen for faster training
- Pre-training random data augmentation block, **GlobalAveragePooling2D**, **Dropout(0.3)**, & **Dense** softmax layers added on top
  - Augmentations: horizontal flips, rotations, zooms, & contrast adjustments applied inside the model
- CNN model setup
  - loss="sparse_categorical_crossentropy"
  - optimizer=tf.keras.optimizers.Adam(1e-3)
  - metrics=["accuracy"]
  - EarlyStopping(monitor="val_loss", patience=4, restore_best_weights=True)
  - ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-5)

VAL MCC: 0.7989435179966928 , ACCURACY: 0.8691176470588236

TEST MCC: 0.7889807439732655 , ACCURACY: 0.864104179794161

Convolutional Neural Network



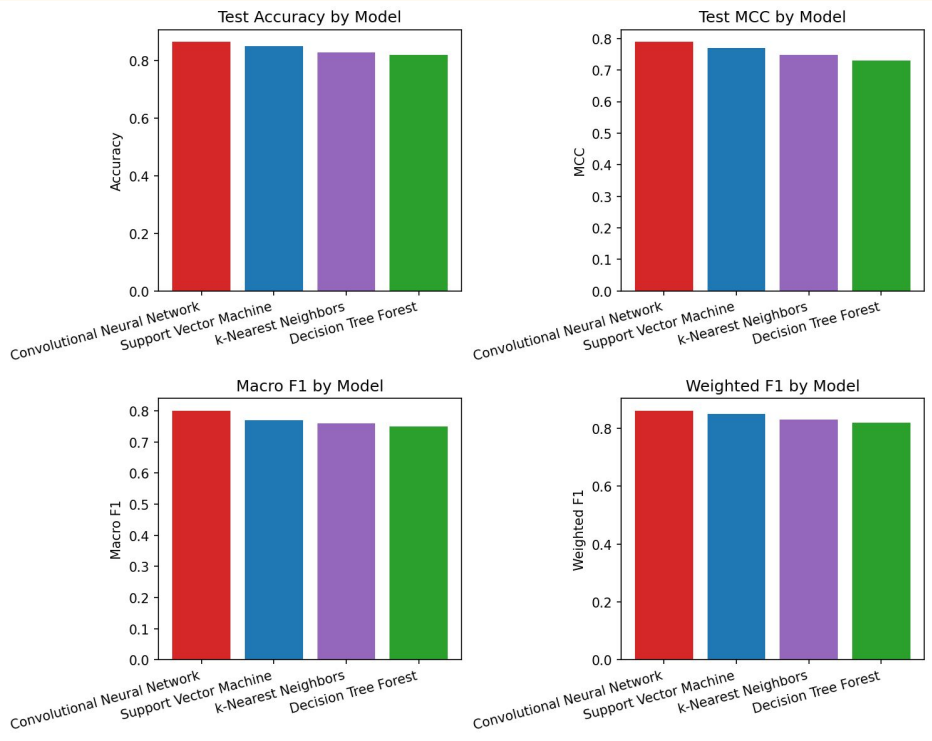


Model Fitting

Classical models (k-NN, SVM, DT) tended to **overfit**, often getting >99% accuracy during training.

- Happened because **dataset** was **too small** compared to how **rich the features** were
- The embeddings are so expressive that the models can **memorize** the training set, but they **lack the regularization & data augmentation tools** that CNNs use to **generalize**

Model Performance Overview

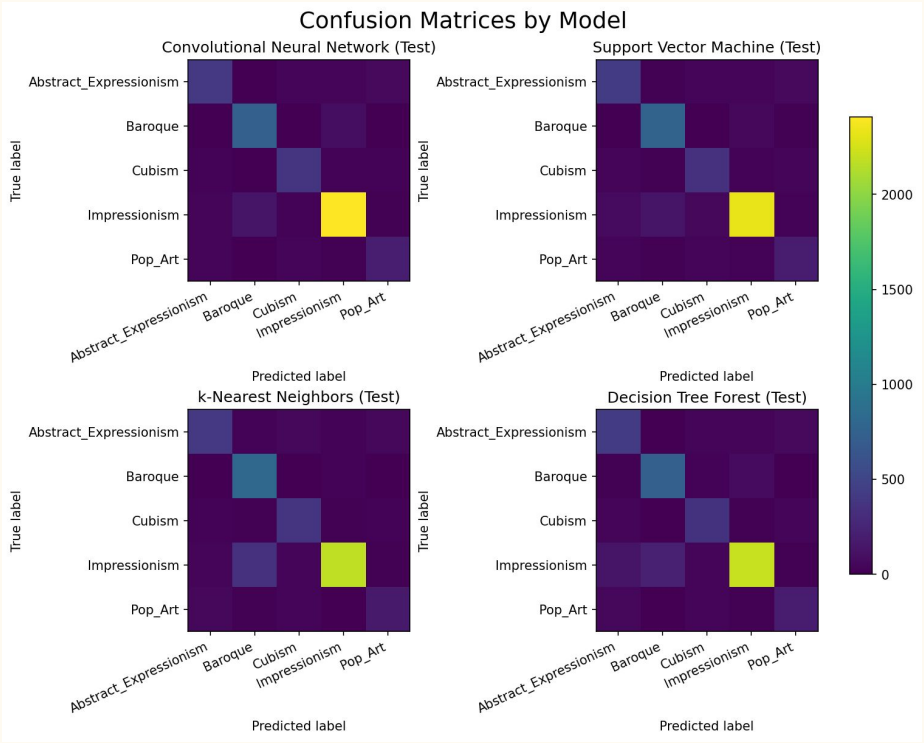


- **CNN** achieved the **strongest** performance overall
  - Highest Val MCC: 0.799
  - Highest Test MCC: 0.789
  - Highest Accuracy (Val/Test): 0.869 / 0.864
  - Best Macro & Weighted F1 across all classes
- **Decision tree forest** was the **weakest** performer

Ranking of Models:

1) Convolutional Neural Network
2) Support Vector Machine
3) k-Nearest Neighbors
4) Decision Tree Forest

Confusion Matrices by Model

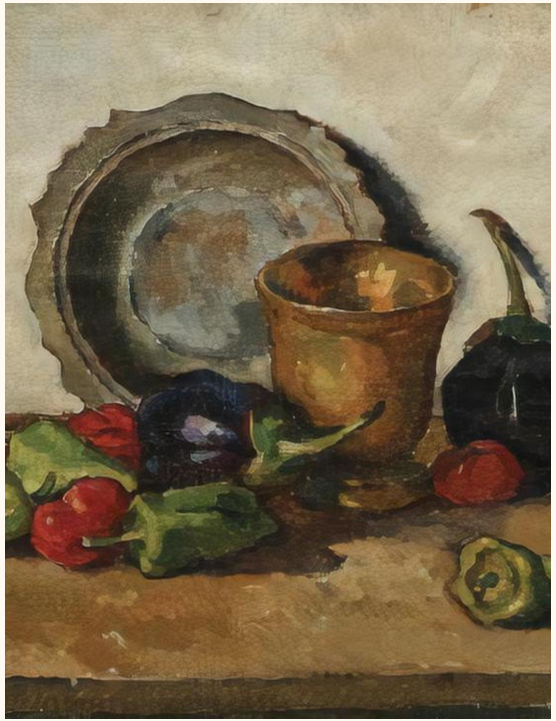


- Majority Impressionism testing samples
- **Pop Art & Abstract Expressionism** are the **most difficult** styles to classify
  - Frequently confused with Cubism & each other
- Impressionism/Baroque confusion hypothesis correct, Cubism/Pop Art not

```
Worst class prediction per model (lowest recall):
- Decision Tree Forest: Pop_Art (recall = 0.630)
- k-Nearest Neighbors: Pop_Art (recall = 0.596)
- Support Vector Machine: Pop_Art (recall = 0.623)
- Convolutional Neural Network: Pop_Art (recall = 0.684)

Top 5 confusions for best model (Convolutional Neural Network):
```

	true_class	pred_class	count
0	Impressionism	Baroque	133
1	Baroque	Impressionism	89
2	Abstract_Expressionism	Pop_Art	59
3	Abstract_Expressionism	Cubism	47
4	Pop_Art	Abstract_Expressionism	44



What does this mean?

- CNN dominated due to learned hierarchical features (edges, textures, shapes, etc.)
- SVM was best out of the classical models due to its performance in high-dimensional spaces
- All models show confusion between visually similar styles
  - Perfect separation/classification is unrealistic because of the blend between styles & overall interpretation

Limitations & Struggles

- The CNN was intentionally limited to offset the bottleneck of computational power .. training took 6+ hours on GPU even with suppression techniques
  - Frozen backbone, EfficientNetB0, mixed precision, etc.
- Some styles contain multiple sub-movements, which may confuse the models
  - Analytical Cubism vs. Synthetic Cubism, Action Painting vs. Color Field Painting

CUBISM

Analytical Cubism

Synthetic Cubism

ABSTRACT EXPRESSIONISM

Action Painting

Color Field Painting

Future Directions

- Art styles are also associated with particular regions, time periods, etc.
  - Contextualization (artist, year, country of origin) could assist learning
- Larger datasets for training to prevent unbalanced testing/increase training diversity
- Expand styles to be classified or add sub-classifications
  - German expressionism, abstract expressionism, neo-expressionism, etc.
  - Could also be influenced by contextualization

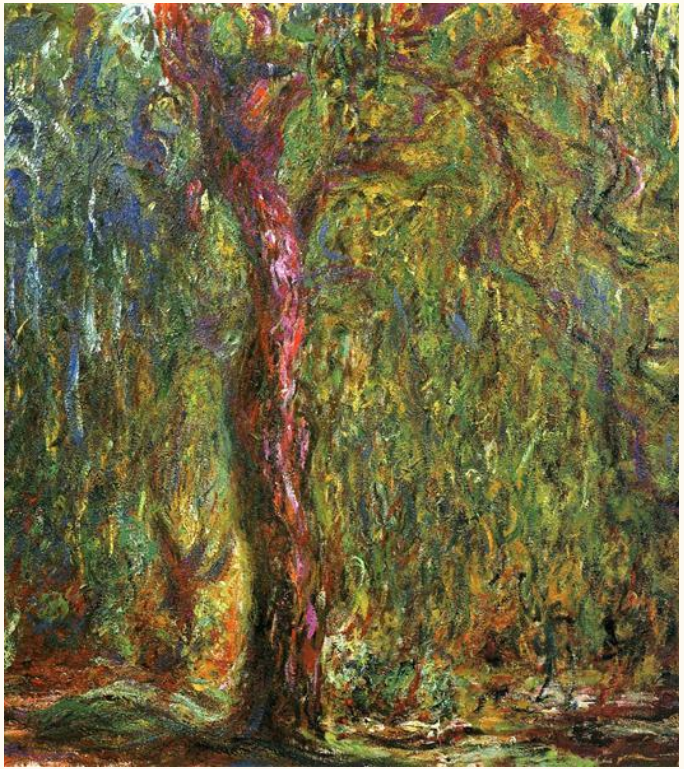
Acknowledgments

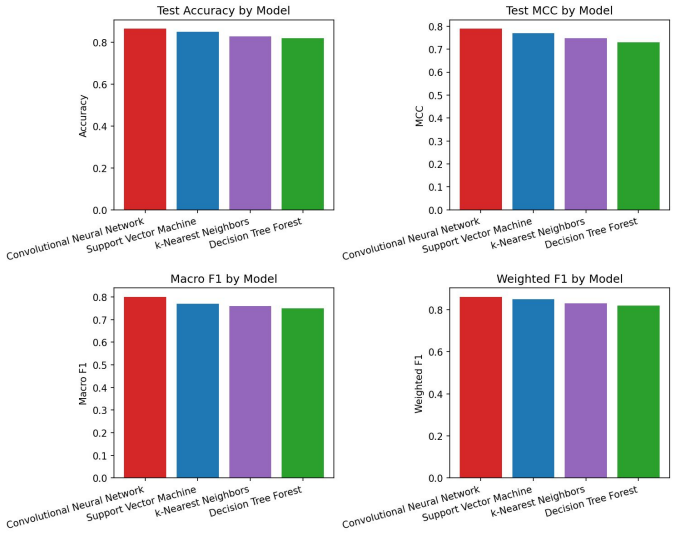
Python Libraries:

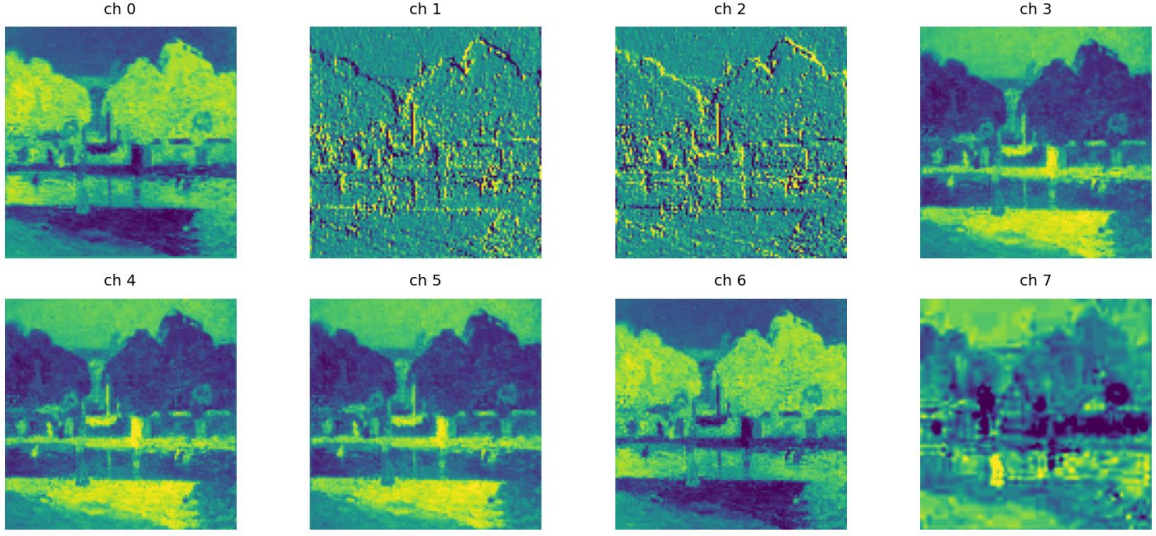
kaggle, pandas, numpy, Pathlib, scikit-learn, scikit-image, opencv-python, tensorflow, matplotlib

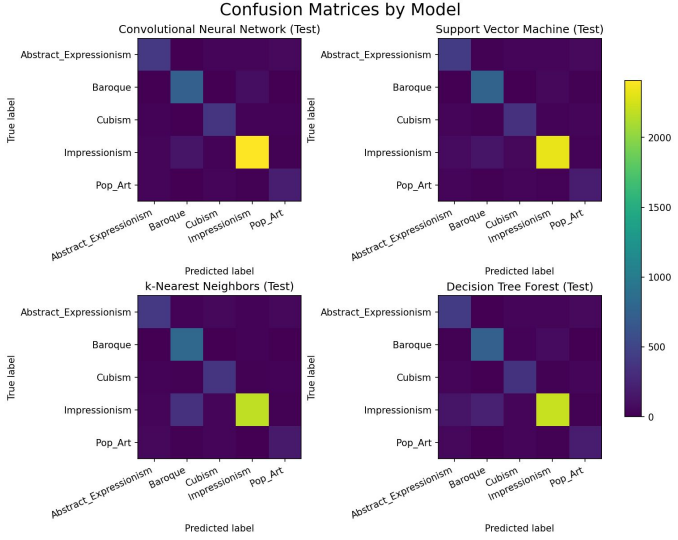
Resources:

- https://scikit-learn.org/stable/modules/tree.html
- https://keras.io/api/applications/
- https://www.quora.com/What-are-the-different-machine-learning-algorithms-for-image-classification-and-why-is-CNN-used-the-most
- https://dev.to/sarvesh42/resnet50-with-tensorflow-implementation-3kfk
- https://www.tensorflow.org/tutorials/images/transfer_learning




Test Accuracy by Model
Accuracy
Convolutional Neural Network
Support Vector Machine
k-Nearest Neighbors
Decision Tree Forest
Test MCC by Model
MCC
Macro F1 by Model
Macro F1
Weighted F1 by Model
Weighted F1


ch 0
ch 1
ch 2
ch 3
ch 4
ch 5
ch 6
ch 7


Confusion Matrices by Model
Convolutional Neural Network (Test)
Support Vector Machine (Test)
k-Nearest Neighbors (Test)
Decision Tree Forest (Test)
Abstract_Expressionism
Baroque
Cubism
Impressionism
Pop_Art
True label
Predicted label